

facendo gli esperimenti visti prima, magari poi vogliamo comunicare questi risultati a qualcuno, o scriverli in un file più piccolo possibile.

- Come facciamo?
- Quanto sarà grande il file finale?

Nell'esempio di prima, se io ripetessi l'esperimento 1 miliardo di volte, mi aspetto che il file più piccolo possibile sia grande 2 miliardi di bit
($2 \text{ bit} \times 1 \text{ miliardo} \dots$)

RAPPRESENTAZIONE DEI RISULTATI

Voglio rappresentare i risultati di un esperimento aleatorio con una stringa di bit (ad esempio per trasmettere o memorizzare in un file...)

DOMANDA: Quanti bit devo usare per rappresentare il risultato? Come devo rappresentare le sequenze di bit?

Vediamo dei casi particolari...

ESEMPIO 1: lancio una moneta.

Un modo di rappresentare i risultati è scrivere 0 se esce T e 1 se esce C

Quindi, uso 1 bit DI RAPPRESENTAZIONE, diverso dal bit DI INFORMAZIONE, per ogni risultato.

ci chiediamo se e' possibile usare meno di un bit.

- Se la moneta e' bilanciata, ogni lancio genera 1 bit di info $\Rightarrow$ Ha senso rappresentare ogni lancio con 1 bit di rappresentazione
- Se la moneta e' fortemente sbilanciata, ogni lancio genera mediamente MOLTO MENO di 1 bit di info $\Rightarrow$ Usare 1 bit di rappresentazione per lancio e' fortemente inefficiente.

Ma quindi, come uso meno di un bit di rappresentazione?

IDEA: Considero più lanci contemporaneamente e codifico un insieme di lanci!

ESEMPIO:

Osservo due lanci di moneta. Supponiamo che $P(T) = p = 3/4$

CODIFICA		PROBABILITA'
TT	0	$(3/4)^2$
TC	10	$3/4 \cdot 1/4$
CT	01	$1/4 \cdot 3/4$
CC	111	$(1/4)^2$

→ Uso più bit di rappresentazione per gli eventi più rari

Vediamo se questa strategia ha aiutato:
calcoliamo il valore atteso della lunghezza della
codifica...

$$E[L] = \left(\frac{3}{4}\right)^2 \cdot 1 + 2 \cdot \left(\frac{3}{4} \cdot \frac{1}{4}\right) \cdot 2 + \left(\frac{1}{4}\right)^2 \cdot 3 = 1,125 \text{ bit}$$

→ Per lancio, stiamo usando $\frac{E[L]}{2} = 0,5625$ bit / lancio

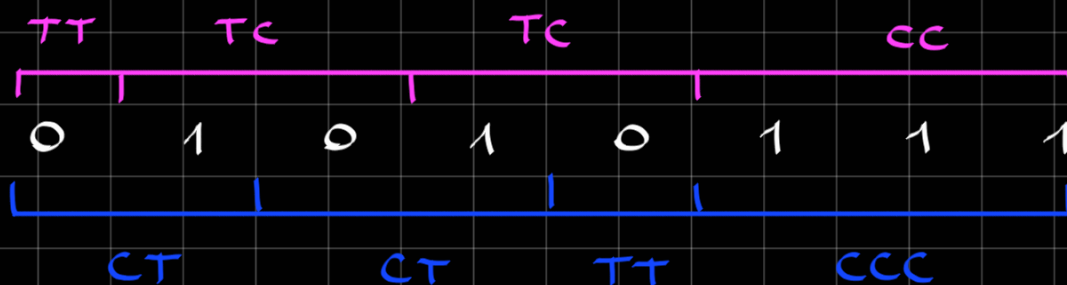
Divido per 2 perché
ogni sequenza di bit è formata da
due lanci

NOMENCLATURA:

- Una particolare sequenza di codifica è
chiamata **PAROLA DI CODICE** (in inglese **CODEWORD**)
- L'insieme di tutte le codeword è chiamata
CODICE (in inglese **CODEBOOK**)

Notiamo un problema con la codifica scritta
prima...

Supponiamo i bit trasmessi siano 01010111.
Posso leggere la sequenza in più modi...



Il problema è che prima abbiamo scelto
una codifica non **UNIVOCA**

Per avere un codice disambiguo, ogni parola di codice non deve essere prefisso di altre parole di codice (CONDIZIONE PREFIX-FREE)

ESEMPIO DI PRIMA: Proviamo a costruire un codice disambiguo. Ecco una possibile codifica...

	CODIFICA	PROBABILITA'
TT	0	$(\frac{3}{4})^2$
TC	1 0	$\frac{3}{4} \cdot \frac{1}{4}$
CT	1 1 0	$\frac{1}{4} \cdot \frac{3}{4}$
CC	1 1 1	$(\frac{1}{4})^2$

In questo caso...

$$E[L] = \left(\frac{3}{4}\right)^2 \cdot 1 + \frac{3}{4} \cdot \frac{1}{4} \cdot 2 + \frac{1}{4} \cdot \frac{3}{4} \cdot 3 + \left(\frac{1}{4}\right)^2 \cdot 4 = 1,6875 \text{ bit}$$

$$\rightarrow \frac{E[L]}{2} = 0,84 \text{ bit/lancio}$$

Il # di bit/lancio resta minore di 1, ma è aumentato rispetto alla codifica non disambigua

DOMANDA: si può fare meglio? Quanto meglio?

Sappiamo che $X \sim \text{Bern}(\frac{3}{4})$

$$\rightarrow H(X) = H_b(\frac{3}{4}) = -\frac{3}{4} \log_2 \frac{3}{4} - \frac{1}{4} \log_2 \frac{1}{4} \approx 0,81 \text{ bit}$$

Quindi, si può migliorare il numero di bit di rappresentazione, ma il limite è $\approx 0,81$ bit di informazione

Dimostriamo che $\frac{E[L]}{\# \text{ lanci}}$ è sempre maggiore di $H(X)$.

DISUGUAGLIANZA DI KRAFT - MCMILLAN

- conviene codificare più lanci di moneta contemporaneamente
- Dovremo usare un codice **PREFIX-FREE** per avere decodificabilità univoca

Non tutti i codici a lunghezza variabile sono prefix-free...

TEOREMA: Posso trovare un codice prefix-free composto da m codewords con lunghezze $l_1, \dots, l_m$ se e solo se ...

$$\sum_{j=1}^m 2^{-l_j} \leq 1$$

← DISUGUAGLIANZA DI KRAFT - MCMILLAN

MORALE: Per lunghezze piccole la disuguaglianza non è verificata $\Rightarrow$ nel mio codebook, devo avere necessariamente parole più lunghe per garantire la disuguaglianza
(Non vedremo la dimostrazione)

CONSEGUENZA: **Non posso abbassare troppo $E[L]$**

ESEMPIO DI PRIMA:

	CODIFICA	PROBABILITA'
TT	0	$(3/4)^2$
TC	10	$3/4 \cdot 1/4$
CT	01	$1/4 \cdot 3/4$
CC	111	$(1/4)^2$

$l_1 = 1; l_2 = 2; l_3 = 2; l_4 = 3$

$$\sum_{j=1}^4 2^{-l_j} = 2^{-1} + 2^{-2} + 2^{-2} + 2^{-3} > 1$$

→ Quindi, **non possiamo trovare un codice prefix-free con queste lunghezze.** Infatti, notiamo che il codebook in questo esempio non è prefix-free!

CODIFICA DI SORGENTE SENZA PERDITA DI INFO
(famoso problema risolto da Shannon nel suo paper del 1946)

DOMANDA: Qual è il minimo valore di $E[L]$?

TEOREMA: CODIFICA DI SORGENTE

Data una v.a. discreta X con m risultati possibili, per ogni codice prefix-free che assegna una codeword di ℓ_j bit al risultato $X = x_j$, si ha che...

$$E[L] = \sum_{j=1}^m \ell_j \cdot p_X(x_j) \geq H(X)$$

(sempre se non vogliamo perdere informazione...)

È una diretta conseguenza di Kraft-McMillan...

ESEMPIO: $X \in \{1, 2, 3, 4\}$

$$p_X(x_j) = \begin{cases} 1/2 & \text{se } j=1 \\ 1/4 & \text{se } j=2 \\ 1/8 & \text{se } j=3, 4 \end{cases}$$

$$\begin{aligned} \rightarrow H(X) &= E\left[\log_2 \frac{1}{p_X(X)}\right] = \\ &= -\frac{1}{2} \log_2 \frac{1}{2} - \frac{1}{4} \log_2 \frac{1}{4} - 2 \cdot \frac{1}{8} \log_2 \frac{1}{8} \approx 1,75 \text{ bit} \end{aligned}$$

Applicando il teo. di codifica di sorgente, possiamo dire che $\exists$ un codice prefix-free tale che...

$$E[L] \geq H(X) = 1,75 \text{ bit}$$

Proviamo a costruire questo codice...

$$x_1 \longrightarrow 0$$

$$x_2 \longrightarrow 10$$

$$x_3 \longrightarrow 110$$

$$x_4 \longrightarrow 111$$

Vale che ...

$$E[L] = 1 \cdot \frac{1}{2} + 2 \cdot \frac{1}{4} + 2 \cdot 3 \cdot \frac{1}{8} = 1,75 \text{ bit}$$

In questo esempio siamo arrivati sul valore esatto dell'entropia. Questo perché ...

$$i(x_j) = \log_2 \frac{1}{p_X(x_j)} = \log_2 (2^{-n_j}) = n_j$$

Numero intero

Dunque quando il valore dell'autoinformazione è un numero intero, non si spreca niente,
 $\rightarrow E[L] = H(X)$

In generale, però, $i(x_j)$ è un numero non intero

ESEMPIO: Se $i(x_j) = 2,05$, il numero minimo di bit di rappresentazione vale 3, altrimenti non si riesce a tenere conto di quello 0,05 di informazione

Al più, sprechiamo 1 bit per ogni risultato x_j , con $j = 1, \dots, m$. Dunque ...

$$H(X) \leq E[L] \leq H(X) + 1 \text{ bit}$$

Quindi, la strategia di prendere $\ell_j = \lceil i(x_j) \rceil$ è quasi ottima!

primo intero strett. maggiore di ...

DIMOSTRIAMO IL TEO. DI CODIFICA DI SORGENTE:
Sappiamo che...

$$\begin{aligned} H(X) &= E \left[\log_2 \frac{1}{p_X(x)} \right] = E \left[\log_2 \frac{2^{\ell(x)}}{2^{\ell(x)} p_X(x)} \right] \\ &= E \left[\log_2 2^{\ell(x)} \right] + E \left[\log_2 \frac{1}{p_X(x) \cdot 2^{\ell(x)}} \right] = \\ &= E[L] + E \left[\log_2 \frac{1}{p_X(x) \cdot 2^{\ell(x)}} \right] \end{aligned}$$

Dobbiamo dimostrare che $E[L] \geq H(X)$.
In altre parole, dobbiamo dimostrare
che ...

$$E \left[\log_2 \frac{1}{p_X(x) \cdot 2^{\ell(x)}} \right] \leq 0$$

Dimostriamo questa disuguaglianza.

Essendoci un valore atteso e un logaritmo,
intuiamo di dover applicare JENSEN...

$$\begin{aligned} E \left[\log_2 \frac{1}{p_X(x) \cdot 2^{\ell(x)}} \right] &\leq \log_2 E \left[\frac{1}{p_X(x) \cdot 2^{\ell(x)}} \right] = \\ &= \log_2 \left(\sum_{j=1}^m p_X(x_j) \cdot \frac{1}{p_X(x_j) \cdot 2^{\ell(x_j)}} \right) = \end{aligned}$$

$$= \log_2 \left(\underbrace{\sum_{j=1}^m 2^{-l(x_j)}} \right)$$

Noi sappiamo che, per Kraft - McMillan, volendo un codice prefix-free deve essere...

$$\sum_{j=1}^m 2^{-l(x_j)} \leq 1$$

$$\Rightarrow \text{Quindi } \log_2 \left(\sum_{j=1}^m 2^{-l(x_j)} \right) \leq 0$$

la disuguaglianza è stata dimostrata

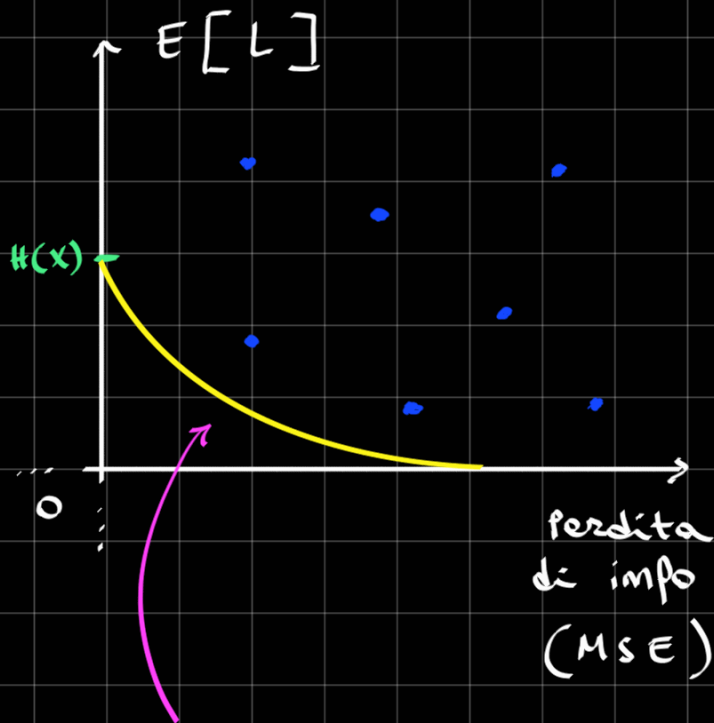


APPLICAZIONE DI QUESTI RISULTATI: la compressione dei file in .zip utilizza simili algoritmi, addirittura senza conoscere l'esperimento aleatorio. ZIP fu creato da LEMPEL - ZIV nel 1977 e l'operazione fatta prende il nome di **COMPRESSIONE UNIVERSALE**

Se la p_x è nota, l'algoritmo che conviene usare è **HUFFMAN CODING**

Tutti questi algoritmi sono di codifica **SENZA PERDITA DI INFORMAZIONI**, perché si usano tutti i bit necessari per non dimenticare niente.

Ma esistono anche algoritmi di codifica CON PERDITA DI INFORM.: un esempio è il formato **JPEG** usato per il trasferimento di immagini



In questo caso, si deve quantificare la quantità di informazione da voler perdere.

Graficamente, possiamo permettere a $E[L]$ di scendere sotto all'entropia $H(X)$!

Si dimostra l'esistenza di una CURVA LIMITE, sotto il quale non si può scendere per il rischio di perdere troppa info.

Questa teoria prende il nome di **RATE-DISTORTION THEORY**

Nella vita reale bisogna considerare anche il **RUMORE** sul canale di comunicazione, e come i pile posso essere corrotti con una certa probabilità.

Questo da vita al **PROBLEMA DELLA CAPACITA' DI COMUNICAZIONE**, da cui segue la formula per cui Shannon è più conosciuto oggi.

quella che abbiamo visto è una teoria dell'informazione **SINTATTICA**, dove si cerca di rappresentare l'informazione "della sorpresa", cioè che dipende solo dalle probabilità

Oggi, si cerca di costruire una teoria dell'info. **SEMANTICA**, dove si dà una certa importanza anche al CONTESTO

Esiste anche l'**INFORMAZIONE QUANTISTICA**, dove si cerca di approssicare la teoria dell'informazione a quella quantistica. Il problema più grande è quello di **QUANTUM ERROR CORRECTION**

(Esiste un corso in magistrale focalizzato su questo argomento.

PREREQUISITI: Algebra lineare e probabilità)
